

DIPLOMATURA EN MACHINE LEARNING CON PYTHON

Algoritmos bayesianos avanzados

Regresión lineal bayesiana

Marco de pensamiento matemático:

Consideremos que tenemos un conjunto de puntos (X_i, Y_i) ya sabemos cómo calcular la línea de regresión si todo lo que tenemos son los puntos. Pero, ¿qué pasaría si, además de los puntos contáramos con algún conocimiento previo de la distribución de probabilidades de la pendiente y la ordenada al origen.

La regresión lineal bayesiana busca estudiar como el conocimiento de los puntos X_i, Y_i modifica las distribuciones pre-existentes (a prior) de la pendiente y la ordenada al origen.

Conociendo esta distribución de probabilidad de las pendientes y ordenadas al origen podemos calcular la distribución de probabilidad de cualquier interpolación o extrapolación entre los X_i y por lo tanto, podremos decidir cuál es la probabilidad de que un dado valor Y corresponda a la serie de puntos para un dado valor de X .

De esta manera podremos estimar si un dado punto tiene más o menos probabilidades que merezca ser explorado o descartado por muy improbable. Este es un lugar oportuno para señalar que un outlier no necesariamente merece ser descartado y olvidado. Según cuál sea nuestro modelo de negocios un outlier puede significar, por ejemplo en el negocio inmobiliario, una casa que se vende a un precio muy conveniente. ¿Es una trampa porque se esconden problemas legales o es una oportunidad porque los dueños se mudan al exterior y venden todo con la mayor rapidez posible?

Esta última pregunta será contestada por los agentes del negocio que analicen la propiedad y no por la estadística bayesiana. Las matemáticas nos ayudarán a decidir en qué medida los puntos son o no remotos respecto de las distribuciones esperables.

Implementación en Python:

Vamos a estar utilizando el paquete pymc3:

- Instalación
- Carga
- Principales funciones
- Ejemplo

Instalación

Según es costumbre, para instalar pymc3 vamos a utilizar pip:

```
Administrador: Windows PowerShell
PS C:\WINDOWS\system32> pip install pymc3
Collecting pymc3
  Downloading pymc3-3.10.0-py3-none-any.whl (1.9 MB)
    | 1.9 MB 2.2 MB/s
Requirement already satisfied: scipy>=0.18.1 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pymc3) (1.6.0)
Requirement already satisfied: pandas>=0.18.0 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pymc3) (1.2.0)
Requirement already satisfied: patsy>=0.5.1 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pymc3) (0.5.1)
Requirement already satisfied: typing-extensions>=3.7.4 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pymc3) (3.7.4.3)
Requirement already satisfied: numpy>=1.13.0 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pymc3) (1.19.3)
Collecting theano-pymc==1.0.11
  Downloading Theano-PyMC-1.0.11.tar.gz (2.8 MB)
    | 2.8 MB ...
Collecting arviz>=0.9.0
  Using cached arviz-0.10.0-py3-none-any.whl (1.5 MB)
Requirement already satisfied: matplotlib>=3.0 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from arviz>=0.9.0->pymc3) (3.3.3)
Requirement already satisfied: setuptools>=38.4 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from arviz>=0.9.0->pymc3) (49.2.1)
Collecting fastprogress>=0.2.0
  Using cached fastprogress-1.0.0-py3-none-any.whl (12 kB)
Requirement already satisfied: python-dateutil>=2.1 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from matplotlib>=3.0->arviz>=0.9.0->pymc3) (2.8.1)
Requirement already satisfied: pyparsing!=2.0.4,!=2.1.2,!=2.1.6,>=2.0.3 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from matplotlib>=3.0->arviz>=0.9.0->pymc3) (2.4.7)
Requirement already satisfied: pillow>=6.2.0 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from matplotlib>=3.0->arviz>=0.9.0->pymc3) (8.1.0)
Requirement already satisfied: cycler>=0.10 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from matplotlib>=3.0->arviz>=0.9.0->pymc3) (0.10.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from matplotlib>=3.0->arviz>=0.9.0->pymc3) (1.3.1)
Requirement already satisfied: six in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages (from cycler>=0.10->matplotlib>=3.0->arviz>=0.9.0->pymc3) (1.15.0)
Requirement already satisfied: pytz>=2017.3 in c:\users\ignac\appdata\local\programs\python\python39\lib\site-packages
(from pandas>=0.18.0->pymc3) (2020.5)
```

La cantidad de pre-requisitos que se van instalando depende de lo que tenga pre-instalado cada sistema operativo.

Cuando ya lo tiene dice: “Requirement already satisfied” y cuando no lo tiene lo descarga y lo instala.

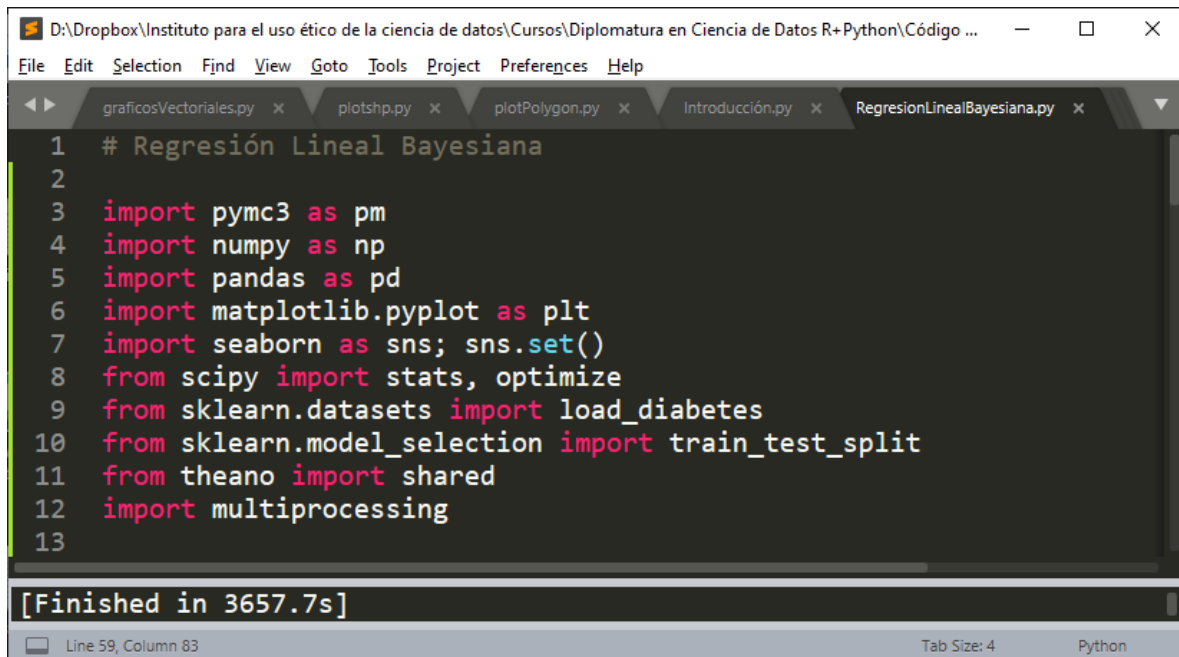
Por ese motivo no hay que preocuparse mientras toda la sesión se mantenga en letras blancas.

Conviene notar que para evitar problemas de permisos esta instalación se hace desde la powershell del administrador.

Para que funcionaran sin problemas las instalaciones de paquetes fue crítico que Python estuviera actualizado a la última versión estable de 64 bits.

Carga de los paquetes:

Para usar estas herramientas desde un programa necesitamos importar las librerías:



The screenshot shows a Python IDE window titled "D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...". The menu bar includes File, Edit, Selection, Find, View, Goto, Tools, Project, Preferences, and Help. The tab bar shows several files: graficosVectoriales.py, plotshp.py, plotPolygon.py, Introducción.py, and RegresiónLinealBayesiana.py. The main editor displays the following code:

```
1 # Regresión Lineal Bayesiana
2
3 import pymc3 as pm
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import seaborn as sns; sns.set()
8 from scipy import stats, optimize
9 from sklearn.datasets import load_diabetes
10 from sklearn.model_selection import train_test_split
11 from theano import shared
12 import multiprocessing
13
```

At the bottom of the editor, a status bar indicates "[Finished in 3657.7s]". The bottom-most status bar shows "Line 59, Column 83", "Tab Size: 4", and "Python".

Principales funciones:

Vamos a utilizar:

`Train_test_split`: sirve para dividir un dataframe en dos trozos al azar para realizar el entrenamiento sobre uno de ellos y la prueba sobre el otro. Es una de las principales formas en las que combatimos el sobre-ajuste.

`Shape`: es un método de los dataframes del paquete pandas. Devuelve una tupla con las dimensiones del dataframe. En nuestro código ese dato no se guarda por lo cual no se hace nada con el.

`shared`: es un método del paquete theano que sirve para que una variable pueda accederse sin problemas desde múltiples instancias de llamadas a función comportándose como una variable global.

`Model`: es un método de pymc3 que invoca el constructor del objeto modelo. Al no haber especificado un modelo (recomendable) nos crea uno lineal.

`Normal`: es un método de pymc3 que sirve para cargar una distribución normal como la correspondiente a las distribuciones a priori de los alpha y betas de nuestro modelo.

`HalfNormal`: es un método de pymc3 que sirve para cargar una distribución lognormal como la correspondiente a las distribuciones a priori del sigma de nuestro modelo.

`find_MAP`: optimiza el modelo a posteriori teniendo en cuenta las distribuciones “a priori” y los datos.

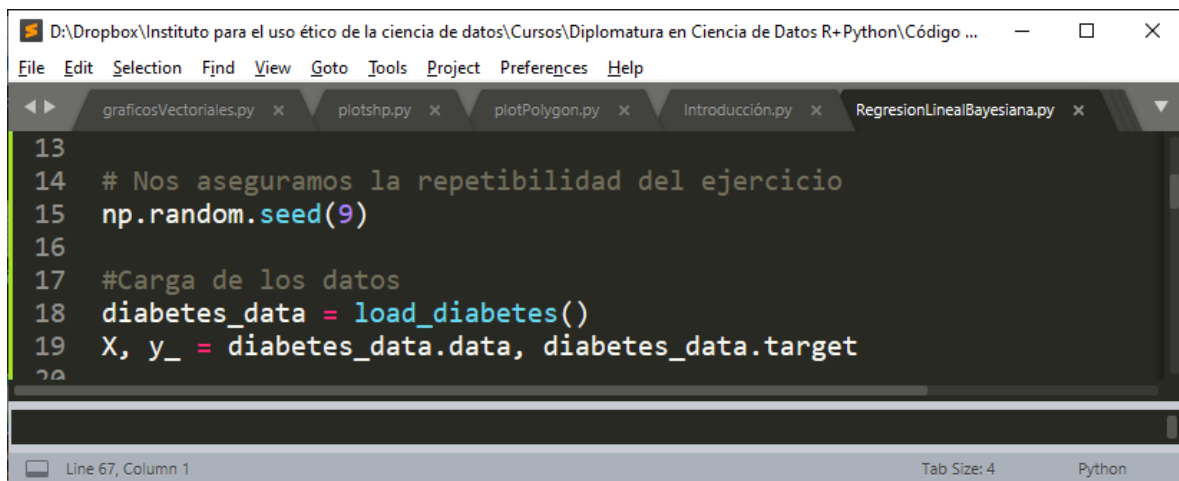
`NUTS`: inicializa la sintonía fina del ajuste del modelo.

sreeze_support: evita que se lancen procesos en distintas hebras.

Sample: toma múltiples muestras de los datos para evaluar la distribución de los parámetros a posteriori. Está preparada para trabajar con múltiples procesadores y ganar así tiempo de ejecución. (Muy conveniente porque es LENTO!)

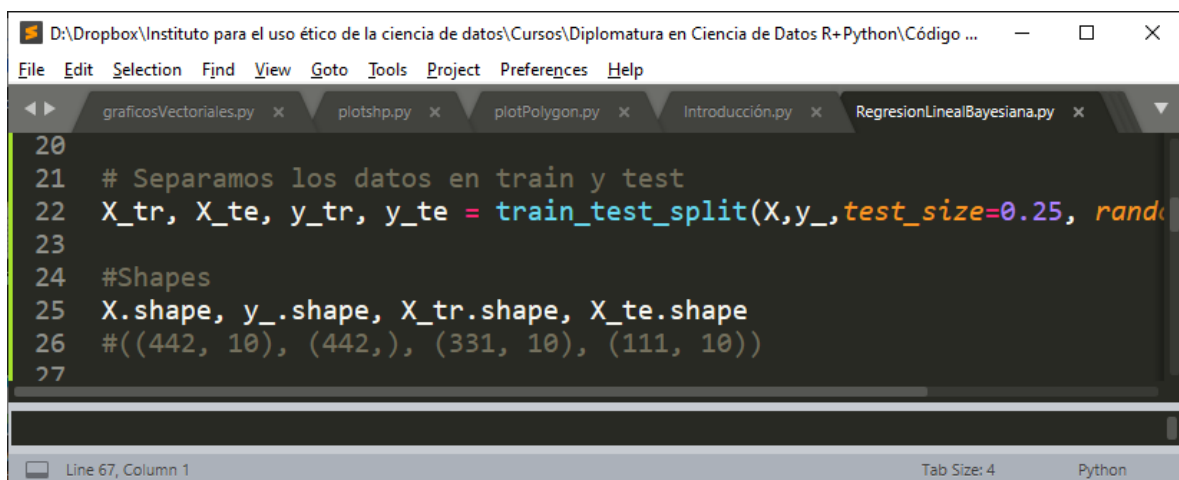
Ejemplo de aplicación:

Empezamos por cargar los datos del dataframe diabetes que nos aporta sklearn:



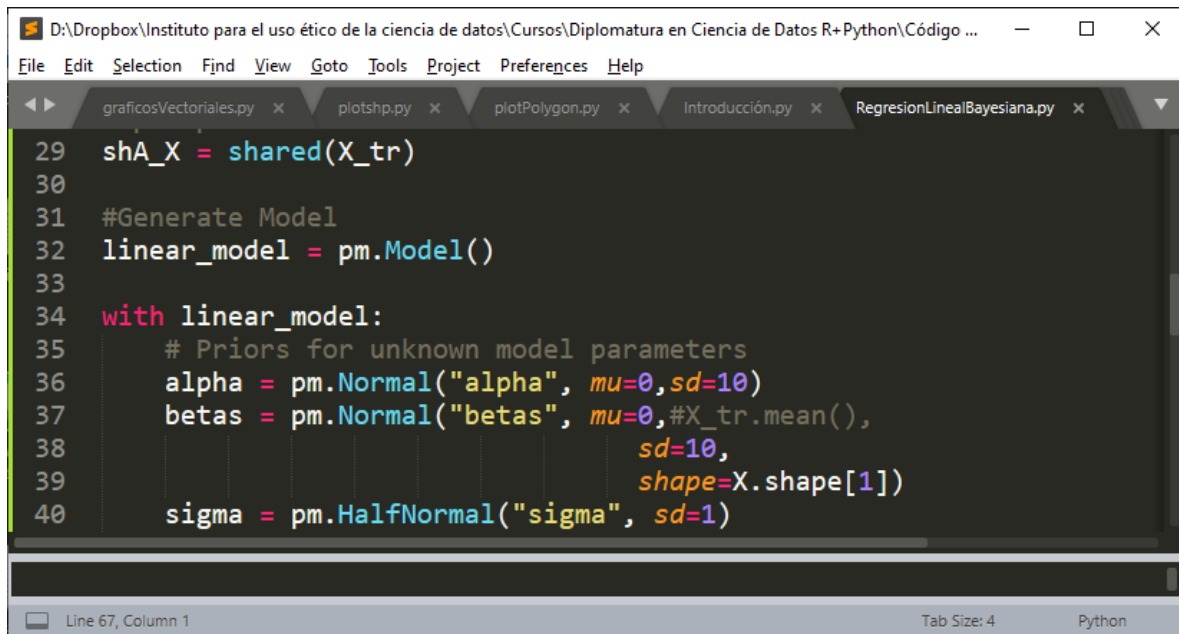
```
13
14 # Nos aseguramos la repetibilidad del ejercicio
15 np.random.seed(9)
16
17 #Carga de los datos
18 diabetes_data = load_diabetes()
19 X, y_ = diabetes_data.data, diabetes_data.target
20
```

Separamos en train y test y verificamos las dimensiones de los distintos conjuntos. (Para verlos por pantalla conviene aplicar un print)



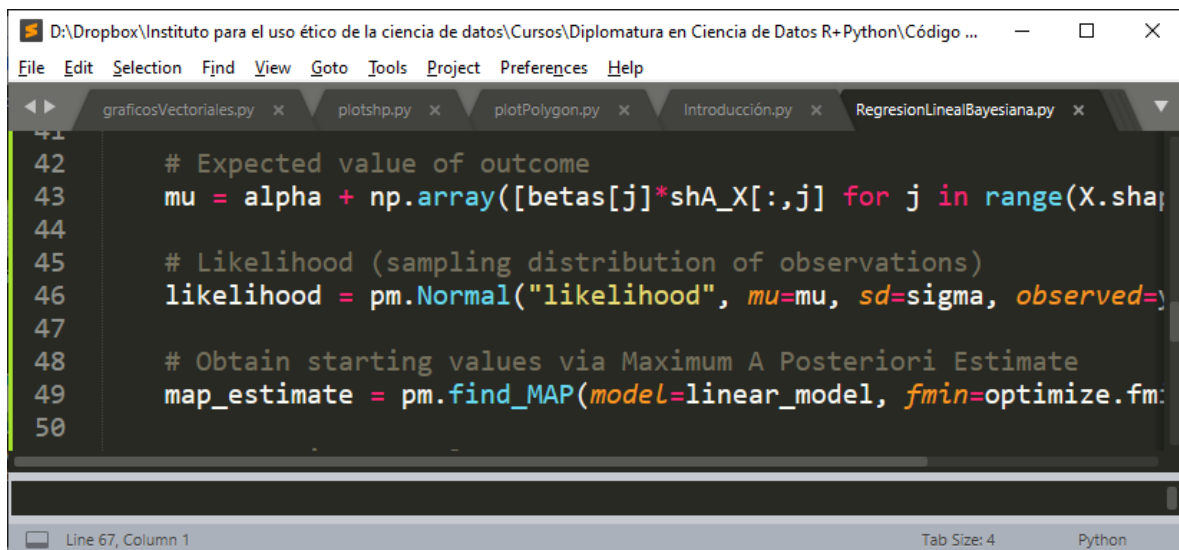
```
20
21 # Separamos los datos en train y test
22 X_tr, X_te, y_tr, y_te = train_test_split(X,y_,test_size=0.25, rando
23
24 #Shapes
25 X.shape, y_.shape, X_tr.shape, X_te.shape
26 #((442, 10), (442,), (331, 10), (111, 10))
27
```

Preparamos los datos, generamos el modelo lineal y le cargamos nuestras distribuciones “a priori”:



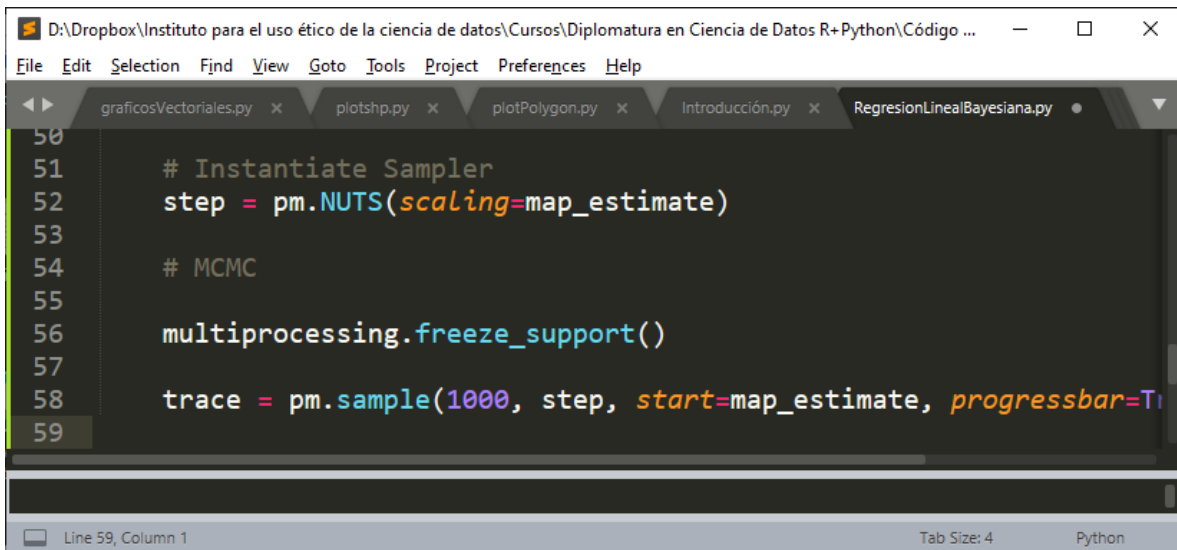
```
29 shA_X = shared(X_tr)
30
31 #Generate Model
32 linear_model = pm.Model()
33
34 with linear_model:
35     # Priors for unknown model parameters
36     alpha = pm.Normal("alpha", mu=0, sd=10)
37     betas = pm.Normal("betas", mu=0, #X_tr.mean(),
38                      sd=10,
39                      shape=X.shape[1])
40     sigma = pm.HalfNormal("sigma", sd=1)
```

Creamos y entrenamos el modelo:



```
42 # Expected value of outcome
43 mu = alpha + np.array([betas[j]*shA_X[:,j] for j in range(X.shape[1])])
44
45 # Likelihood (sampling distribution of observations)
46 likelihood = pm.Normal("likelihood", mu=mu, sd=sigma, observed=y)
47
48 # Obtain starting values via Maximum A Posteriori Estimate
49 map_estimate = pm.find_MAP(model=linear_model, fmin=optimize.fmin)
50
```

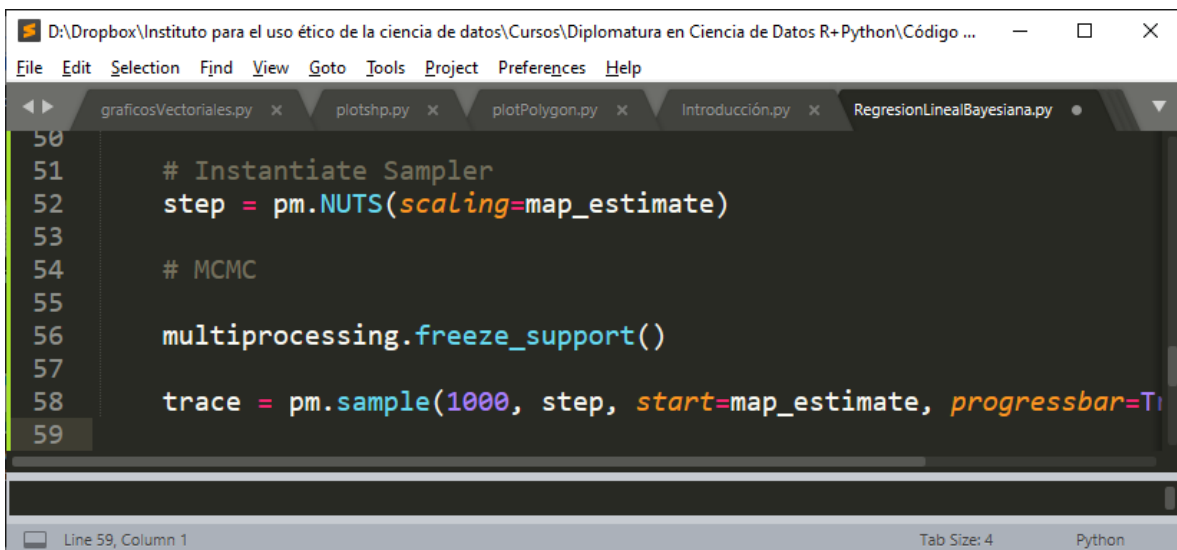
Generamos ahora 1000 muestras de los datos para explorar la distribución de los parámetros:



```
50
51 # Instantiate Sampler
52 step = pm.NUTS(Scaling=map_estimate)
53
54 # MCMC
55
56 multiprocessing.freeze_support()
57
58 trace = pm.sample(1000, step, start=map_estimate, progressbar=True)
59
```

Line 59, Column 1 Tab Size: 4 Python

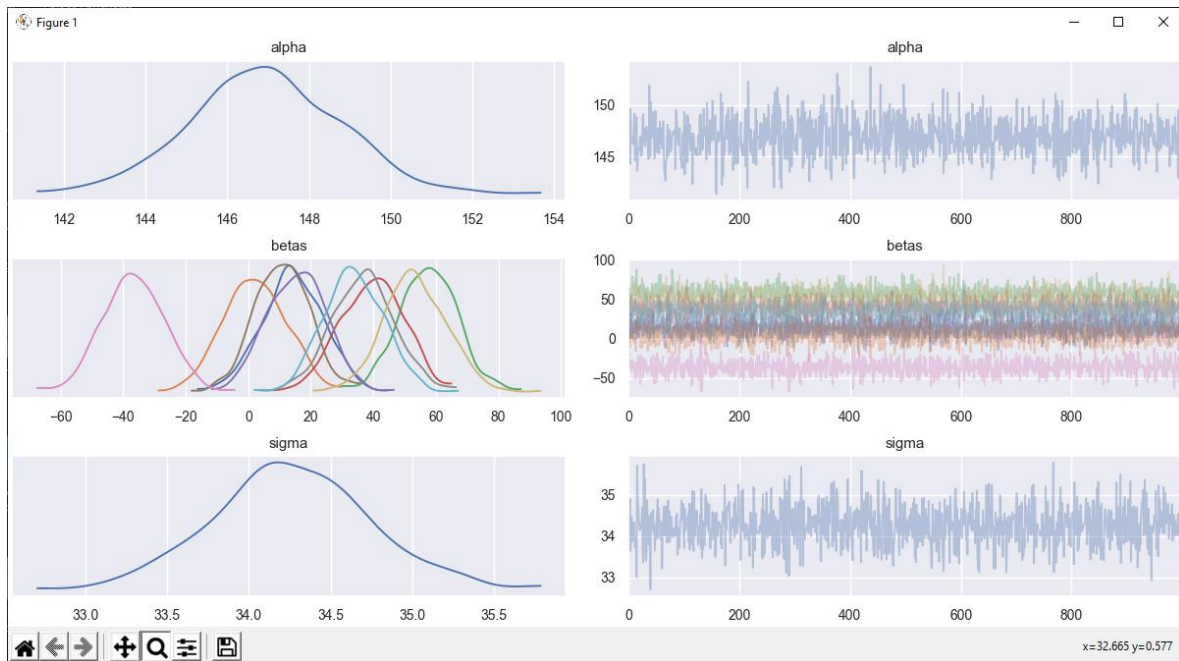
Por fin podemos mostrar los resultados conseguidos con:



```
50
51 # Instantiate Sampler
52 step = pm.NUTS(Scaling=map_estimate)
53
54 # MCMC
55
56 multiprocessing.freeze_support()
57
58 trace = pm.sample(1000, step, start=map_estimate, progressbar=True)
59
```

Line 59, Column 1 Tab Size: 4 Python

Y finalmente obtenemos:



Redes Bayesianas

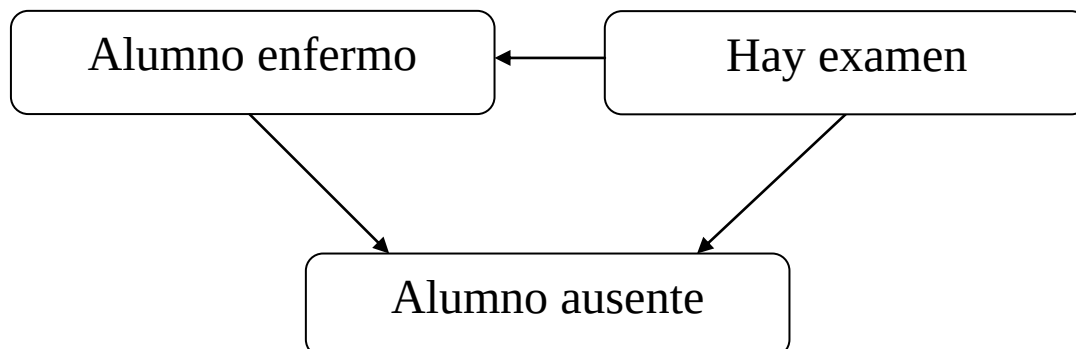
Se trata de un instrumento que nos permite relacionar probabilidades de ocurrencia de causas y consecuencias mediante una red de probabilidades condicionales.

En términos matemáticos lo vamos a representar como un grafo acíclico y dirigido.

Cada nodo del grafo representará una variable aleatoria que podrá ser continua o discreta. Cada segmento del grafo contendrá la información de probabilidad condicional que vincula ambas variables aleatorias.

Dos nodos que no están vinculados por un segmento se consideran condicionalmente independientes.

Ejemplo súper simple:



Hoy es día de examen.

Un alumno no vino.

¿Estará enfermo?

Una posibilidad es que se trate de una falta casual por otro motivo que no se relaciona ni con su salud ni con el examen.

Otra alternativa es que casualmente se haya enfermado.

También podría ocurrir que el examen lo haya enfermado (lo que algunos autores llaman un fenómeno psicosomático)

No podemos tampoco descartar que haya faltado por el examen sin que sufra ningún problema de salud.

¿Cómo podemos abordar el problema?

Empezamos por identificar las variables aleatorias:

- Alumno enfermo
- Hay examen
- Alumno ausente

El segundo paso es calcular las probabilidades condicionales:

- Probabilidad(Enfermo/Examen)
- Probabilidad(Enfermo/No Examen)
- Probabilidad(No Enfermo/Examen)
- Probabilidad(No Enfermo/No Examen)

- Probabilidad(Ausente/Examen)
- Probabilidad(Ausente/No Examen)
- Probabilidad(No Ausente/Examen)
- Probabilidad(No Ausente/No Examen)

- Probabilidad(Ausente/Enfermo)

- Probabilidad(Ausente/No Enfermo)
- Probabilidad(No Ausente/Enfermo)
- Probabilidad(No Ausente/No Enfermo)

¿De dónde sacamos esos valores?

Una posibilidad es la historia.

Supongamos que tenemos una tabla llamada Asistencias que da cuenta de la asistencia de nuestro hipotético alumno:

Fecha	Ausente	Enfermo	Examen
1	NO	SI	NO
2	NO	NO	NO
3	SI	SI	NO
4	NO	SI	NO
5	NO	NO	NO
6	SI	SI	SI
7	NO	SI	SI
8	NO	NO	NO
9	SI	SI	NO
10	SI	NO	SI

Para calcular la probabilidad tenemos que hacer algo tan difícil como contar. Por ejemplo para la probabilidad de no ausente dado no enfermo contamos:

- A. Cuantos días no estuvo enfermo: Na
- B. Cuantos días no estuvo enfermo y no estuvo ausente: Nb

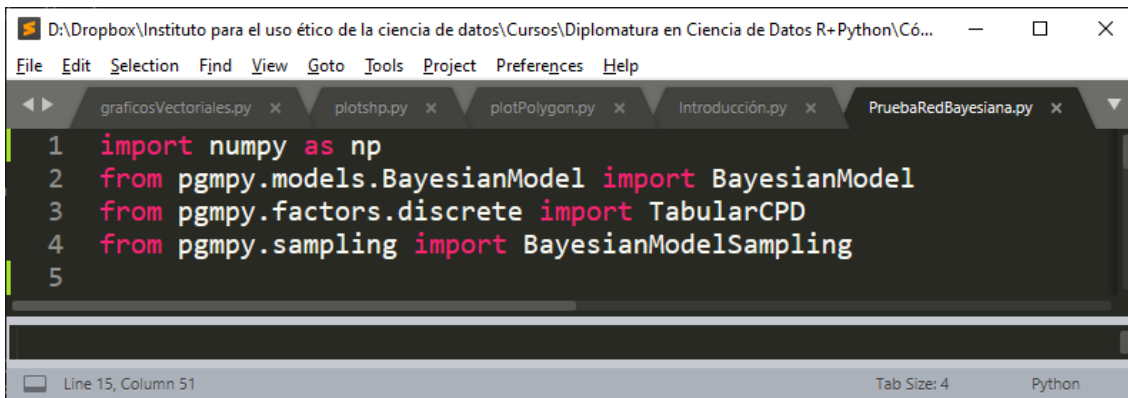
Luego:

Probabilidad(No ausente dado que no enfermo) = Nb/Na

Y, de esta manera, podemos calcular todas las probabilidades condicionales.

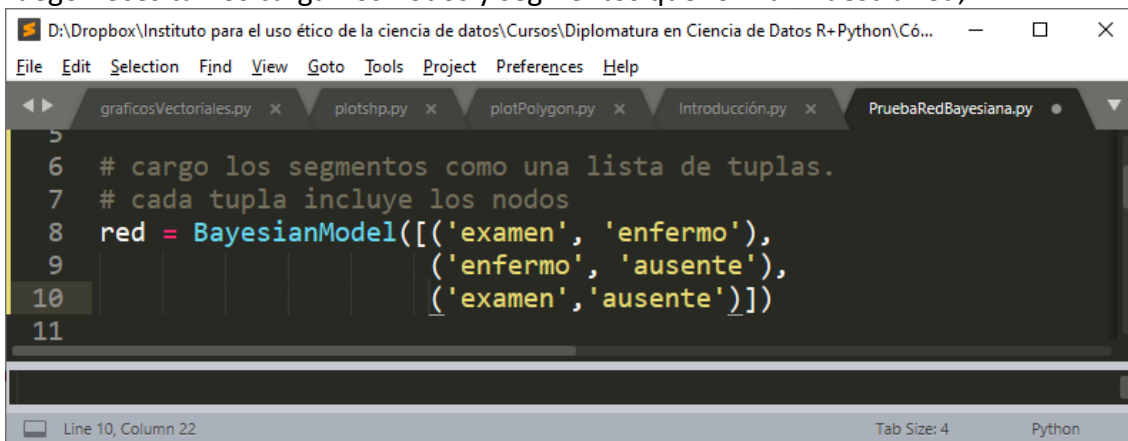
Paquete pgmpy:

Vamos a utilizar pgmpy para crear redes bayesianas y realizar inferencias
El primer paso es importar el paquete:



```
1 import numpy as np
2 from pgmpy.models.BayesianModel import BayesianModel
3 from pgmpy.factors.discrete import TabularCPD
4 from pgmpy.sampling import BayesianModelSampling
5
```

Luego necesitamos cargar los nodos y segmentos que forman nuestra red;



```
6 # cargo los segmentos como una lista de tuplas.
7 # cada tupla incluye los nodos
8 red = BayesianModel([('examen', 'enfermo'),
9                     ('enfermo', 'ausente'),
10                    ('examen', 'ausente')])
11
```

Nos toca ahora cargar las probabilidades condicionales. Para eso vamos a usar la clase TabularCPD a la que necesitamos introducirle los siguientes parámetros:

1. Nombre del nodo: examen, enfermo o ausente
2. Cardinalidad de la variable: 2, 2, 2 pues las tres variables tienen sólo dos estados
3. Probabilidades de los estados

En nuestro caso tenemos que empezar por el nodo que no depende de nada que es el de examen y cargamos sus probabilidades totales: (SI: 0,3 y NO: 0,7)

Luego vamos por las probabilidades condicionales.

Para el nodo enfermo, que depende sólo de examen cargamos dos listas de dos valores cada una:

- La primera lista corresponde a las probabilidades de enfermo = SI para las dos posibilidades de la variable examen.
- La segunda lista corresponde a las probabilidades de enfermo = NO para las dos posibilidades de la variable examen.

El nodo ausente es el más complicado de cargar.

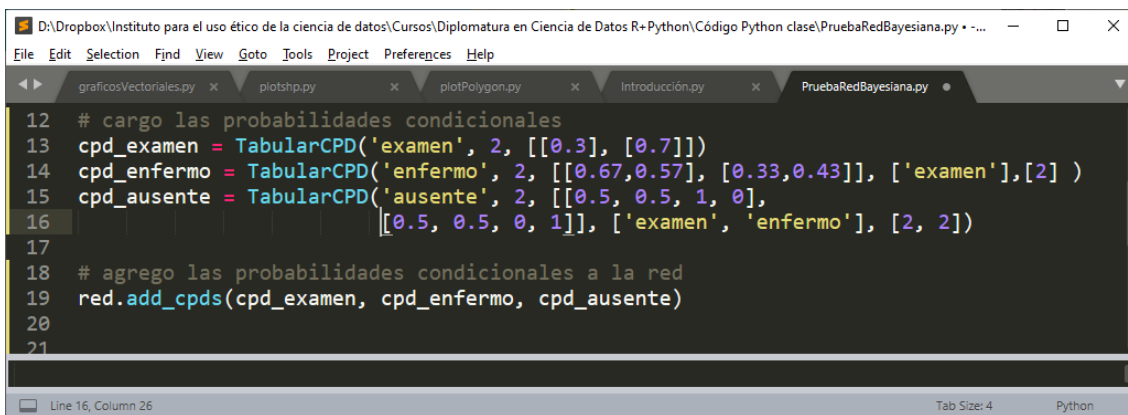
Vamos a incluir una lista que va a contener dos elementos.

Cada elemento será una lista de cuatro valores.

El primer elemento tendrá las probabilidades de ausente = SI para las cuatro combinaciones de examen, enfermo: SI-SI, SI-NO, NO-SI, NO-NO

El segundo elemento tendrá las probabilidades de ausente = NO para las cuatro combinaciones de examen, enfermo: SI-SI, SI-NO, NO-SI, NO-NO

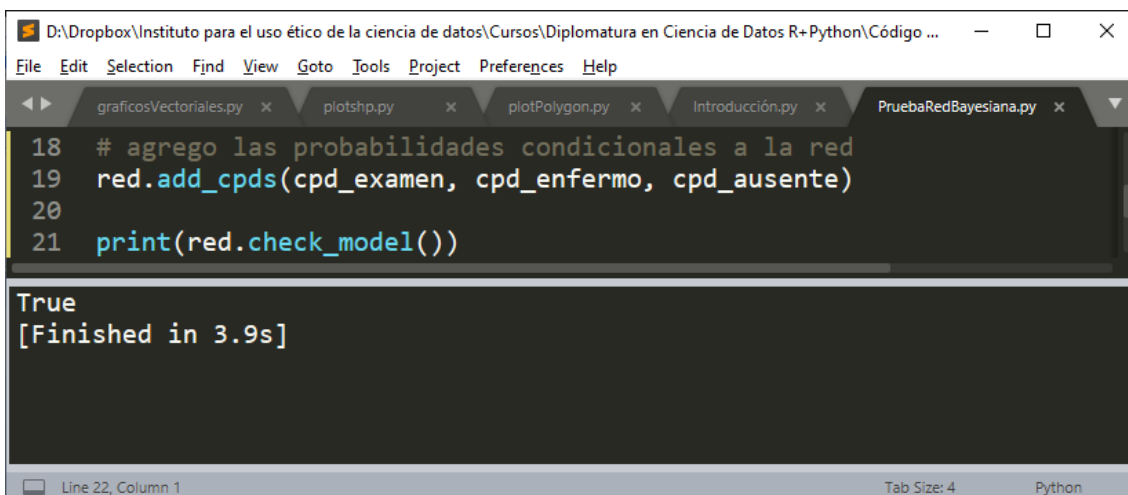
Luego la carga nos queda como:



```
12 # cargo las probabilidades condicionales
13 cpd_examen = TabularCPD('examen', 2, [[0.3], [0.7]])
14 cpd_enfermo = TabularCPD('enfermo', 2, [[0.67, 0.57], [0.33, 0.43]], ['examen'], [2] )
15 cpd_ausente = TabularCPD('ausente', 2, [[0.5, 0.5, 1, 0],
16                                           [0.5, 0.5, 0, 1]], ['examen', 'enfermo'], [2, 2])
17
18 # agrego las probabilidades condicionales a la red
19 red.add_cpds(cpd_examen, cpd_enfermo, cpd_ausente)
20
21
```

Ya hemos cargado los nodos. ¿Está completa nuestra red?

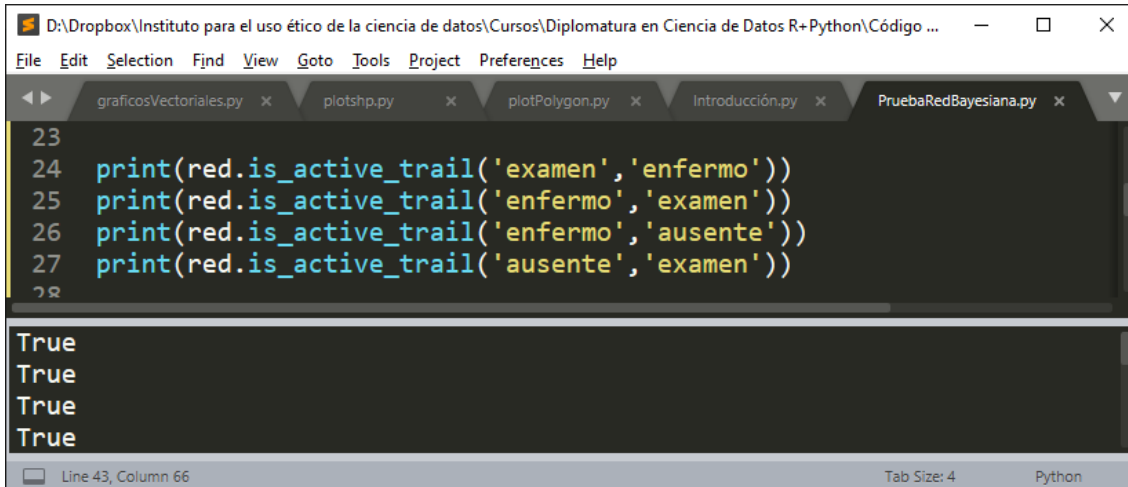
El paquete nos ofrece la función `check_model()` para que podamos verificarlo:



```
18 # agrego las probabilidades condicionales a la red
19 red.add_cpds(cpd_examen, cpd_enfermo, cpd_ausente)
20
21 print(red.check_model())
```

True
[Finished in 3.9s]

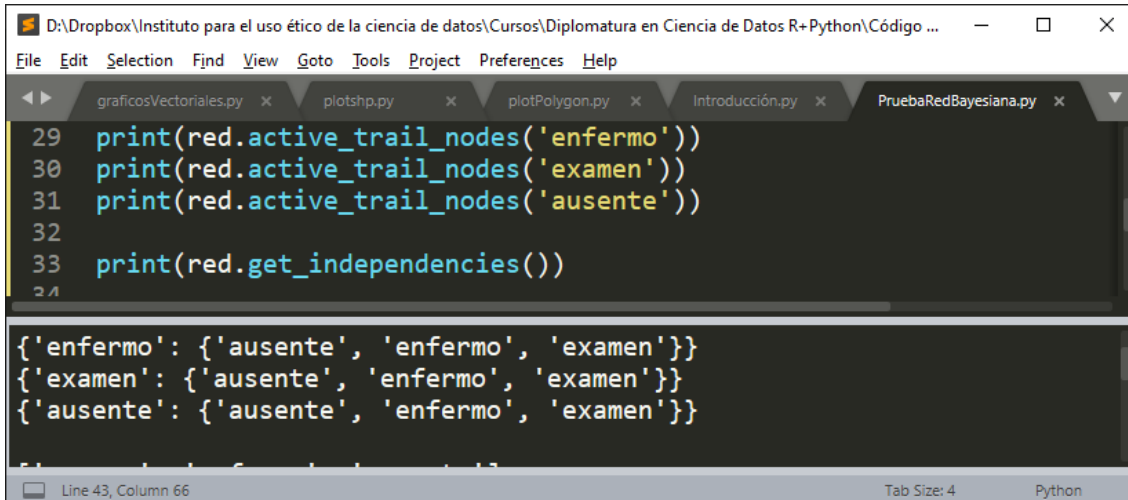
Una de las cosas que nos pueden pasar entre los nodos de una red es que se condicionen unos a otros. Eso lo verificamos con:



```
D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...
File Edit Selection Find View Goto Tools Project Preferences Help
graficosVectoriales.py x plotshp.py x plotPolygon.py x Introducción.py x PruebaRedBayesiana.py x
23
24 print(red.is_active_trail('examen','enfermo'))
25 print(red.is_active_trail('enfermo','examen'))
26 print(red.is_active_trail('enfermo','ausente'))
27 print(red.is_active_trail('ausente','examen'))
28

True
True
True
True
Line 43, Column 66 Tab Size: 4 Python
```

Otra forma de verlo es:

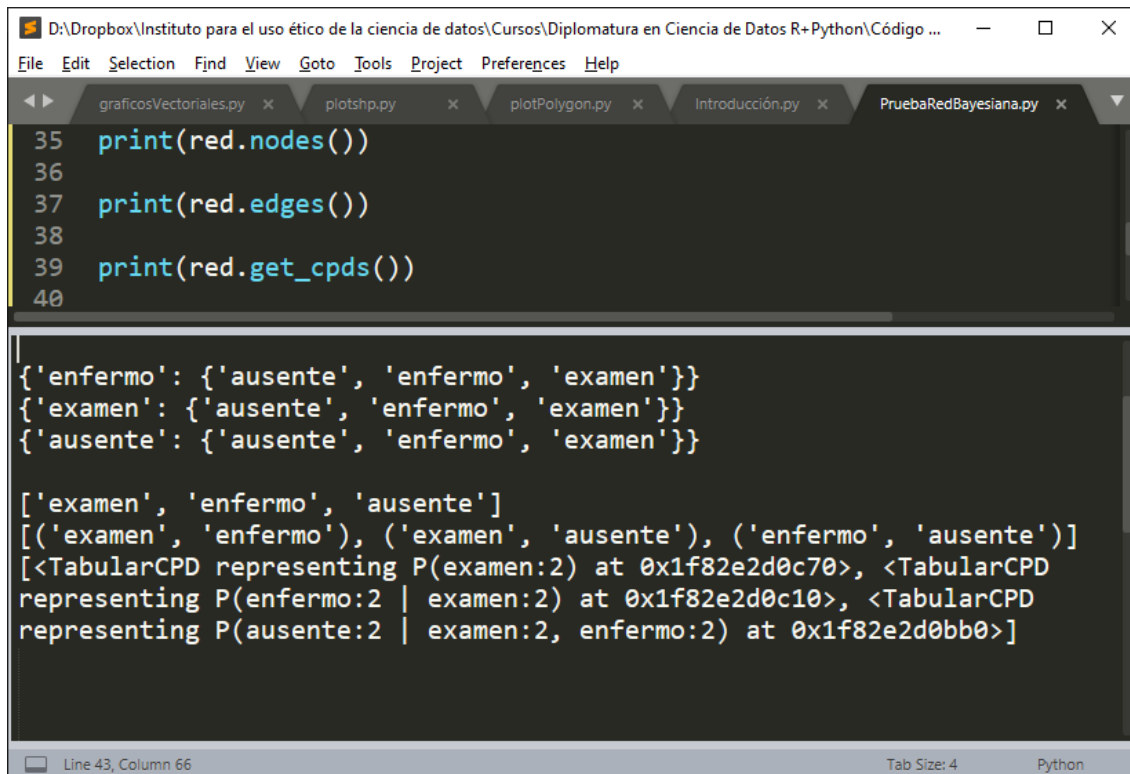


```
D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...
File Edit Selection Find View Goto Tools Project Preferences Help
graficosVectoriales.py x plotshp.py x plotPolygon.py x Introducción.py x PruebaRedBayesiana.py x
29 print(red.active_trail_nodes('enfermo'))
30 print(red.active_trail_nodes('examen'))
31 print(red.active_trail_nodes('ausente'))
32
33 print(red.get_independencies())
34

{'enfermo': {'ausente', 'enfermo', 'examen'}}
{'examen': {'ausente', 'enfermo', 'examen'}}
{'ausente': {'ausente', 'enfermo', 'examen'}}
Line 43, Column 66 Tab Size: 4 Python
```

Notamos que la línea 33 nos produce un renglón en blanco. Eso se debe a que en nuestra red bayesiana NO HAY nodos que sean independientes.

Contamos con facilidades para visualizar los nodos, los segmentos y las tablas cpd:



```
D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...
File Edit Selection Find View Goto Tools Project Preferences Help
graficosVectoriales.py x plotshp.py x plotPolygon.py x Introducción.py x PruebaRedBayesiana.py x
35 print(red.nodes())
36
37 print(red.edges())
38
39 print(red.get_cpds())
40

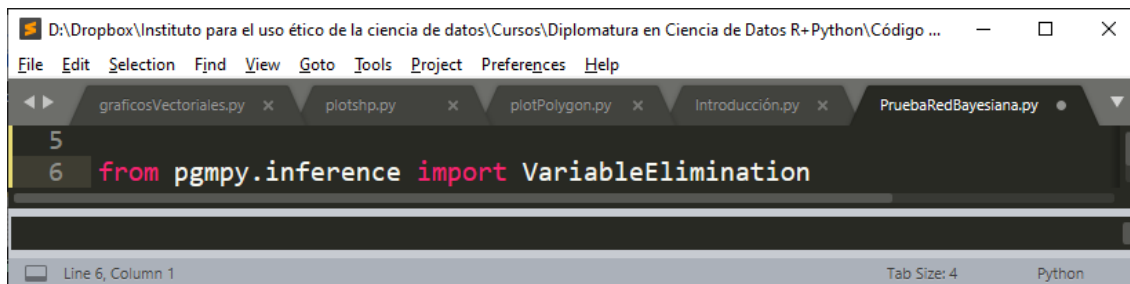
{'enfermo': {'ausente', 'enfermo', 'examen'}}
{'examen': {'ausente', 'enfermo', 'examen'}}
{'ausente': {'ausente', 'enfermo', 'examen'}}

['examen', 'enfermo', 'ausente']
[('examen', 'enfermo'), ('examen', 'ausente'), ('enfermo', 'ausente')]
[<TabularCPD representing P(examen:2) at 0x1f82e2d0c70>, <TabularCPD
representing P(enfermo:2 | examen:2) at 0x1f82e2d0c10>, <TabularCPD
representing P(ausente:2 | examen:2, enfermo:2) at 0x1f82e2d0bb0>]
```

Line 43, Column 66 Tab Size: 4 Python

Estamos entonces listos para hacer inferencias.

Dentro de la zona de import deberemos incluir:



```
D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...
File Edit Selection Find View Goto Tools Project Preferences Help
graficosVectoriales.py x plotshp.py x plotPolygon.py x Introducción.py x PruebaRedBayesiana.py
5
6 from pgmpy.inference import VariableElimination
```

Line 6, Column 1 Tab Size: 4 Python

Y para hacer la consulta recurrimos a:

```

41
42 red_infer = VariableElimination(red)
43
44 q = red_infer.query(variables = ['examen'], evidence={'ausente':1})
45
46 print(q)

```

```

0%|          | 0/1 [00:00<?, ?it/s]
Finding Elimination Order: : 0%|          | 0/1 [00:00<?, ?it/s]

0%|          | 0/1 [00:00<?, ?it/s]

Eliminating: enfermo: 0%|          | 0/1 [00:00<?, ?it/s]
Eliminating: enfermo: 100%|          | 1/1 [00:00<00:00, 998.41it/s]
+-----+-----+
| examen | phi(examen) |
+-----+-----+
| examen(0) | 0.3326 |
+-----+-----+
| examen(1) | 0.6674 |
+-----+-----+

Finding Elimination Order: : 100%|          | 1/1 [00:00<00:00, 45.57it/s]
[Finished in 1.5s]

```

¿Cómo interpretamos el resultado?

Al decirle 'ausente':1 le estamos informando a la red un hecho: el alumno con el que acumulamos los datos faltó.

Y la estamos interrogando acerca de las probabilidades de que haya un examen hoy.

También podríamos asegurarle a la red de que hay un examen hoy y ver las probabilidades de que nuestro alumno falte:

```

D:\Dropbox\Instituto para el uso ético de la ciencia de datos\Cursos\Diplomatura en Ciencia de Datos R+Python\Código ...
File Edit Selection Find View Goto Tools Project Preferences Help
graficosVectoriales.py x plotshp.py x plotPolygon.py x Introducción.py x PruebaRedBayesiana.py x
48 q = red_infer.query(variables = ['ausente'],evidence={'examen':1})
49
50 print(q)
51

0%|          | 0/1 [00:00<?, ?it/s]

Eliminating: enfermo: 0%|          | 0/1 [00:00<?, ?it/s]
Eliminating: enfermo: 100%|██████████| 1/1 [00:00<00:00, 1036.65it/s]
+-----+-----+
| ausente | phi(ausente) |
+=====+=====+
| ausente(0) | 0.5700 |
+-----+-----+
| ausente(1) | 0.4300 |
+-----+-----+

Finding Elimination Order: : 100%|██████████| 1/1 [00:00<00:00, 40.10it/s]
Finding Elimination Order: : 100%|██████████| 1/1 [00:00<00:00, 45.59it/s]
[Finished in 1.4s]

Line 51, Column 1
Tab Size: 4
Python

```